

Backend Engineering — Revised Edition

Phases 1 through 5: OOP fundamentals, the Core Java toolkit, Spring Boot & REST APIs, database engineering with PostgreSQL/MongoDB/Redis, and a new Production Readiness phase covering observability, resilience, Docker, and event-driven basics.

At a Glance	
Phases	1–5
Topics	35
Estimated Duration	17–20 weeks
Difficulty	Beginner → Intermediate
Suggested Pace	8–12 hrs/week · 2–3 topics/week (part-time default)
Projects Included	2–5 detailed hands-on projects per topic, step-by-step
Documentation Sources	2+ official/primary sources per topic

What Changed in This Revision

This edition implements a full curriculum review against professional backend-engineering practice. **[NEW]** marks added topics, **[UPDATED]** marks deepened or reordered ones. Unmarked topics are carried over from the original roadmap unchanged.

Reordered. Concurrency & Multithreading moved to after Java Streams in Phase 2, so async/functional idioms (CompletableFuture) build on Streams instead of preceding them.

Reordered. Request Validation & Exception Handling now precedes REST API Design in Phase 3, so the single global error-response shape is built once and reused, instead of being drafted twice.

Added. Records & Sealed Classes topic at the end of Phase 1 — modern Java idioms used throughout Spring Boot DTOs and domain modeling.

Deepened. Concurrency topic now includes CompletableFuture, virtual threads (Project Loom), and ConcurrentHashMap/atomics alongside the original thread-pool and race-condition projects.

Deepened. Streams topic adds an explicit parallelStream() project, including when NOT to use it.

Deepened. JWT Authentication topic adds CORS configuration and an OAuth2/OIDC awareness project, so hand-rolled JWT is presented alongside the delegated-auth pattern most companies actually run in production.

Added. API Documentation with OpenAPI/Swagger — new topic in Phase 3.

Added. Caching with Redis — new topic in Phase 4.

Deepened. Testing Spring Boot Applications topic adds a Testcontainers project, replacing H2/mocks with a real containerized Postgres for integration tests.

Added. Phase 5 — Production Readiness & Cloud-Native Basics: Observability & Actuator, Resilience & Rate Limiting, Containerization with Docker, and an awareness-level Messaging & Events topic. Scoped deliberately to stay short of full microservices architecture.

Updated. Final project checklists for Phases 3, 4, and 5 now include OpenAPI docs, Actuator health checks, Redis caching, Testcontainers, and a docker-compose stack.

Updated. At-a-Glance figures, topic count, and progress tracker revised to match the new phase and topic totals.

Topics Covered

PHASE 1 — OOP Fundamentals

Encapsulation · Inheritance · Abstraction · Polymorphism · Composition · Exceptions · SOLID Principles & Design Patterns · Records & Sealed Classes (NEW)

PHASE 2 — Core Java Toolkit

2D & 3D Arrays · Java Collections Framework · Generics · Exception Handling — Deep Dive · File I/O · Java Streams · Concurrency & Multithreading (reordered + deepened) · JDBC · Build Tools · Testing (JUnit & Mockito) · Logging

PHASE 3 — Framework & API

Spring Core & Dependency Injection · Request Validation & Exception Handling (reordered) · REST API Design · API Documentation with OpenAPI/Swagger (NEW) · JWT Authentication (deepened with CORS + OAuth2 awareness) · GraphQL · Testing Spring Boot Applications

PHASE 4 — Databases

JPA Entity Mapping & Relationships · Lazy vs Eager Loading & the N+1 Problem · JPQL & Native Queries · PostgreSQL Indexing & Query Optimization · Transactions · MongoDB & Document Databases · Caching with Redis (NEW) · Schema Migrations (Flyway)

PHASE 5 — Production Readiness & Cloud-Native Basics (NEW)

Observability & Actuator · Resilience & Rate Limiting · Containerization with Docker · Messaging & Events (Awareness Level)

Next: once ShopFlow's backend API is live, tested, containerized, and production-hardened, move on to Part B — Frontend Engineering.

PHASE 1 OF 5

OOP Fundamentals

Encapsulation · Inheritance · Abstraction · Polymorphism · Composition · Exceptions · SOLID · Design Patterns · Records & Sealed Classes

The goal of this phase is active recall through code, not passive reading. For each pillar below, build several of the suggested projects in IntelliJ before moving on — the point is to force yourself to write the pattern from memory, not to build a complete app.

Encapsulation

Hiding internal state behind a controlled public interface, so a class's data can only change through methods that validate it.

Subtopics: private fields · public getters/setters · validation inside setters · immutability basics

1. Bank Account — private balance, public deposit/withdraw with validation

Make balance private → add deposit(amount) rejecting negatives → add withdraw(amount) rejecting overdrafts → add getBalance() → test edge cases in main()

2. Student Grade Book — private grades list exposed only through safe methods

Make a private List grades → add addGrade() validating 0-100 → add getAverage() and getHighest() → never expose the raw list

3. Immutable Point Class — practice full immutability, not just private fields

Make x and y private final → set both only via constructor → provide no setters → add a translate() that returns a NEW Point instead of mutating

Study Resources

- Oracle: Classes and Objects (Encapsulation)
- Baeldung: Encapsulation in Java

Inheritance

Letting one class reuse and extend another's fields and behavior, modeling an is-a relationship.

Subtopics: extends keyword · super() calls · method overriding · protected access

1. Animal Hierarchy — Animal base class with Dog, Cat, Bird subclasses

Write an abstract-ish Animal with name and speak() → extend with Dog, Cat, Bird → override speak() and move() per subclass → call super() in each constructor

2. Employee Payroll — Employee base with FullTime, PartTime, Contractor

Write Employee with calculatePay() → extend with 3 subclasses → override calculatePay() differently per type

Study Resources

- Oracle: Inheritance (Java Tutorials)
- Baeldung: Inheritance in Java

Abstraction

Exposing only the essential behavior through an abstract class or interface, hiding implementation detail the caller doesn't need.

Subtopics: abstract classes · abstract methods · interfaces · programming to an interface

1. Payment Gateway — abstract Payment with CreditCard, GCash, PayMaya

Write abstract class Payment with abstract process() → implement 3 subclasses → write a Checkout class using only the Payment type

2. Notification System — Notifier interface with Email, SMS, Push

Define interface Notifier with send(String msg) → implement 3 notifiers → write client code that only depends on Notifier

Study Resources

- Oracle: Interfaces and Abstract Classes
- Baeldung: Abstract Classes in Java

Polymorphism

Calling the same method name on different object types and getting behavior specific to each.

Subtopics: method overriding · dynamic dispatch · upcasting · the @Override annotation

1. Discount Engine — one apply(price) call, three discount types

Define DiscountStrategy interface → implement Regular/Senior/StudentDiscount → store in a List → loop and print each discounted price

2. Game Character Moves — Warrior, Mage, Archer each override attack()

Write abstract Character with abstract attack() → extend with 3 subclasses → write a game loop calling attack() on any Character

Study Resources

- Oracle: Polymorphism (Java Tutorials)
- Baeldung: Polymorphism in Java

Composition

Building complex objects out of smaller ones (has-a) instead of inheritance (is-a) — often the more flexible choice.

Subtopics: has-a relationships · composing objects · delegation · composition vs inheritance

1. Computer Builder — Computer has-a CPU, RAM, Storage, GPU

Write small classes for each part → compose them as fields inside Computer → build 2 different configs by swapping components

2. Car Engine Swap — prove composition's flexibility over inheritance

Write an Engine interface with start() → compose an Engine field inside Car → implement GasEngine and ElectricEngine → swap engines at runtime

Study Resources

- GeeksforGeeks: Composition in Java
- Refactoring.Guru: Inheritance vs Composition

Exceptions

Signaling and handling error conditions with a structured, typed alternative to error codes or silent failure.

Subtopics: try/catch/finally · custom exception classes · throwing vs catching · exception hierarchies

1. ATM Simulator — custom exceptions for real banking error cases

Define `InsufficientFundsException`, `InvalidPINException`, `DailyLimitExceededException` → throw each from the right validation point → catch and handle distinctly

2. Resource Cleanup Demo — guarantee cleanup even when something throws

Open a fake resource → force an exception mid-processing → use `try-with-resources` to guarantee it closes

Study Resources

- Oracle: Exceptions (Java Tutorials)
- Baeldung: Exception Handling in Java

SOLID Principles & Design Patterns

Apply SOLID principles across everything you write. Implement at least one pattern per category below.

Subtopics: Single Responsibility · Open/Closed · Liskov Substitution · Interface Segregation · Dependency Inversion · Creational patterns · Structural patterns · Behavioral patterns

1. Factory Method: `ProductFactory` — centralize object creation logic

Define a `Product` interface → write 2-3 concrete types → write `ProductFactory.create(type)` → confirm callers never use ``new`` directly

2. Strategy: `DiscountStrategy` Swap — change an algorithm at runtime without `if/else`

Reuse the `DiscountStrategy` interface from Polymorphism → inject a different strategy into the same `Cart` instance

3. Decorator: `Coffee Order Add-ons` — add behavior without subclass explosion

Write a base `Coffee` class with `cost()` → write a decorator abstract class wrapping a `Coffee` → stack decorators and confirm `cost()` adds up correctly

Study Resources

- Refactoring.Guru: Design Patterns
- Baeldung: SOLID Principles in Java

Records & Sealed Classes (Modern Java) [NEW]

Java 14+ record types and Java 17+ sealed classes/interfaces — now idiomatic for DTOs and closed domain hierarchies, and expected reading in any current Spring Boot codebase.

Subtopics: record type basics · compact constructors & validation · sealed interfaces + permits · pattern matching for switch

1. DTO Conversion Pass — replace 3 old Phase-1 classes with records

Pick 3 immutable classes you already wrote (e.g. `Point`, an `Address`) → rewrite each as a record → add a compact constructor for validation → confirm `equals()`/`hashCode()`/`toString()` now come for free

2. Sealed `Payment Result` — model a closed set of outcomes

Define a sealed interface `PaymentResult` permitting `Success`, `Declined`, `Error` records → write a `switch` expression handling all three exhaustively → confirm the compiler flags a missing case

Study Resources

- Oracle: Record Classes
- Oracle: Sealed Classes and Interfaces
- Baeldung: Java Records

Final project for this phase — ShopFlow: Foundation (Mini E-Commerce Core)

- Product hierarchy: PhysicalProduct, DigitalProduct (inheritance + polymorphism)
- CartItem composes Product + quantity + discount (composition)
- DiscountStrategy interface: PercentOff, FixedOff, NoDiscount (Strategy pattern)
- ProductFactory.create(type) to instantiate correct subclass (Factory Method)
- Private cart total, expose getTotal() only (encapsulation)
- Custom exceptions: OutOfStockException, InvalidQuantityException
- NEW: model the checkout outcome as a sealed PaymentResult, and represent the CartItem line as a record where it stays a pure value

Build a console-based shopping cart system that forces you to apply everything from this phase. No Spring Boot yet — pure Java.

PHASE 2 OF 5

Core Java Toolkit

Collections · Generics · Exceptions · File I/O · Streams · Concurrency · JDBC · Build Tools · Testing · Logging

This is the layer most self-taught devs skip, then feel shaky about in interviews. Everything here is what Spring Boot, JPA, and your own future code leans on constantly. Note the reorder: Concurrency now follows Streams, so async patterns like CompletableFuture build on functional idioms you've just practiced, instead of being introduced cold.

2D & 3D Arrays

Arrays of arrays — the foundation for grids, matrices, and board-shaped data before you ever reach for a Collections-based structure.

Subtopics: declaring `int[][]/int[][][]` · nested loops for traversal · jagged arrays · matrix operations

1. Matrix Addition & Multiplication — classic 2D array math

Declare two `int[][]` matrices → write `addMatrices()` → write `multiplyMatrices()` with the row-by-column algorithm

2. Tic-Tac-Toe Board — a 2D array as game state

Represent the board as `char[3][3]` → write `placeMove(row,col,player)` → write `checkWinner()` scanning rows/cols/diagonals

Study Resources

- Oracle: Arrays (Java Tutorials)
- Baeldung: Multi-Dimensional Arrays in Java

Java Collections Framework

The standard library's ready-made data structures — List, Set, Map, Queue — that replace most hand-rolled array logic.

Subtopics: List (ArrayList, LinkedList) · Set (HashSet, TreeSet) · Map (HashMap, TreeMap) · Queue/Deque · Iterator · Comparable vs Comparator · time complexity per structure (NEW)

1. Inventory Manager — store and look up items two ways

Store products in a List → also index them in a Map by ID → compare List vs Map lookup speed conceptually

2. Task Scheduler — PriorityQueue ordering by urgency

Define a Task class with a priority field → add tasks to a PriorityQueue → poll tasks off in priority order

3. Big-O Lookup Table (NEW) — document the complexity of every structure you used

For ArrayList, LinkedList, HashMap, TreeMap, HashSet, ArrayDeque: write get/insert/delete/search complexity from memory → verify each against the Javadoc → keep this as a standing reference sheet

Study Resources

- Oracle: Collections Framework Overview
- Baeldung: Guide to the Java ArrayList

Generics

Writing type-safe, reusable classes and methods that work across many types without casting.

Subtopics: generic classes · bounded types · wildcards (? extends / ? super) · generic interfaces

1. Generic Repository Interface — preview what Spring Data will do for you later

Define interface Repository → add save(T entity), findById(ID id), findAll() → implement it in-memory with a Map

2. Generic Stack Implementation — a reusable stack for any type

Write class Stack backed by an ArrayList → add push(), pop(), peek(), isEmpty() → test with Integers and Strings

Study Resources

- Oracle: Generics (Java Tutorials)
- Baeldung: Java Generics

Exception Handling — Deep Dive

Beyond try/catch basics: the mechanics that matter for production code and interviews.

Subtopics: checked vs unchecked · try-with-resources · multi-catch · exception chaining

1. Exception Chaining Logger — preserve the original cause

Catch a low-level exception → wrap it in a custom exception via `new CustomEx(msg, cause)` → log the full chain with getCause()

2. Checked vs Unchecked Refactor — convert one to the other and feel the difference

Write a method throwing a checked exception → refactor to a RuntimeException subclass instead → discuss which is better here

Study Resources

- Baeldung: Checked vs Unchecked Exceptions
- Baeldung: try-with-resources

File I/O

Reading and writing data outside the running program — text files, CSVs, and config files.

Subtopics: java.io vs java.nio.file · BufferedReader/Writer · reading/writing CSV · serialization basics

1. CSV Import/Export — round-trip your product catalog through a file

Load product data from a CSV with BufferedReader → parse into Product objects → write back out with BufferedWriter

2. Directory Walker — use java.nio.file.Files to explore a folder tree

Use Files.walk() on a project directory → filter to only .java files → print the 3 largest files by size

Study Resources

- Oracle: File I/O (java.nio.file)

- Baeldung: Java - Write to File

Java Streams

A functional-style pipeline for processing collections — source, then a chain of operations, then a result.

Subtopics: *map/filter/sorted/distinct/flatMap · Collectors (toList, groupingBy, joining) · reduce · parallelStream() and when NOT to use it (NEW) · when to use a plain loop instead*

1. Product Query Pipeline — filter, map, and sort in one chain

Start from a List → filter() by category → map() to a lighter DTO → sorted() by price, collect() to a List

2. Sales Report Grouping — groupingBy + summing in one line

Start from a List → groupingBy(category) → downstream collector summingDouble(price)

3. Parallel vs Sequential Benchmark (NEW) — see when parallelStream helps and when it hurts

Run the same aggregation over a small collection (~100 items) with stream() then parallelStream(), timing both → repeat over a large collection (~5,000,000 items) → observe parallelStream only wins at scale and adds overhead on small/I/O-bound work → write 2 bullet points on when to reach for it

Study Resources

- Oracle: java.util.stream (Javadoc)
- Baeldung: Java 8 Streams Introduction
- Baeldung: Parallel Streams

Concurrency & Multithreading [UPDATED]

Beyond thread-pool basics: the modern concurrency toolkit a 2026 backend job actually expects, now sequenced after Streams so the functional style is already second nature.

Subtopics: *Thread vs Runnable · ExecutorService & thread pools · race conditions · synchronized keyword · CompletableFuture (NEW) · virtual threads / Project Loom (NEW) · ConcurrentHashMap & atomics (NEW)*

1. Parallel File Processor — use a thread pool instead of raw threads

Create an ExecutorService with a fixed pool → submit one task per file → collect results with Future → shut down cleanly

2. Race Condition Demo, then Fix It — reproduce the bug on purpose

Share one int counter across 100 threads incrementing it unsynchronized → observe the wrong final count → add synchronized → confirm it's now correct → note the performance trade-off

3. CompletableFuture Chain (NEW) — compose async work without blocking

Write 2 methods that each return a slow value → wrap each in supplyAsync() → combine them with thenCombine() → add exception handling with exceptionally() → compare readability to nested callbacks

4. Virtual Threads Benchmark (NEW) — feel why Loom matters for I/O-bound services

Submit 10,000 short blocking tasks (simulated I/O sleep) to a platform-thread pool, time it → resubmit the same 10,000 tasks to Executors.newVirtualThreadPerTaskExecutor() → compare thread count and completion time → write a note on why this matters for a REST API under load

Study Resources

- Oracle: Concurrency (Java Tutorials)
- Baeldung: Java ExecutorService Guide
- Baeldung: CompletableFuture
- OpenJDK JEP 444: Virtual Threads

JDBC

Raw, unabstracted database connectivity — do this before Spring Data/Hibernate so you see exactly what JPA will later hide from you.

Subtopics: DriverManager & Connection · PreparedStatement · ResultSet mapping · transactions (commit/rollback)

1. Raw JDBC CRUD — hand-write every SQL operation

Connect to PostgreSQL with DriverManager → write INSERT via PreparedStatement → write SELECT and map ResultSet rows to objects

2. SQL Injection Demo (then fix it) — see the risk, then remove it

Build a query with raw string concatenation → demonstrate an injection payload breaking it → rewrite using PreparedStatement placeholders

Study Resources

- Oracle: JDBC Basics
- Baeldung: Introduction to JDBC
- PostgreSQL JDBC Driver Documentation

Build Tools

Maven and Gradle — you'll use one of these on every Java job, since Spring Boot itself is generated from a template built on one of them.

Subtopics: Maven (pom.xml) · Gradle (build.gradle) · dependency management · standard project layout

1. Convert to Maven — restructure a plain-Java project properly

Create a standard src/main/java, src/test/java layout → write a pom.xml → add one real dependency → build with mvn package

2. Multi-Module Maven Project — practice a realistic multi-module layout

Create a parent pom with two modules (core and api) → let api depend on core → build from the parent

Study Resources

- Apache Maven: Getting Started Guide
- Baeldung: Introduction to Maven

Testing (JUnit & Mockito)

Writing automated tests that prove your code works — and keep proving it after every change.

Subtopics: JUnit 5 (@Test, assertions) · Mockito (@Mock, when/thenReturn) · Arrange-Act-Assert · unit vs integration tests

1. Mock the Repository — test a service in isolation

Write a Service depending on a Repository interface → mock it with @Mock → stub with when().thenReturn() → verify() it was called exactly once

2. Parameterized Tests — run one test logic against many inputs

Use @ParameterizedTest with @CsvSource → test a validation method against 5 inputs → confirm both valid and invalid cases are covered

Study Resources

- JUnit 5 User Guide
- Mockito official documentation
- Baeldung: Mockito Series

Logging

Replacing `println` debugging with real, leveled, configurable logging — the way production code actually reports what's happening.

Subtopics: log levels (TRACE/DEBUG/INFO/WARN/ERROR) · SLF4J + Logback · log patterns/formatting · what never to log

1. Add Logging to ShopFlow — replace every `println` with real logging

Add SLF4J + Logback → replace `println` with `logger.info()/debug()` → add `logger.error()` with the exception on every catch block

2. Redact-Sensitive-Data Filter — practice never logging secrets

Write a method that logs a user object → confirm it would log a password field → mask it with a `DTO/toString()` override

Study Resources

- SLF4J User Manual
- Logback documentation

Final project for this phase — ShopFlow: Core Java Service Layer

- Rebuild the cart's data layer using Collections (Map catalog) and a 2D-array-backed inventory grid
- Generic Repository implemented in-memory, ready to be swapped for JPA later
- Full custom exception hierarchy (ShopFlowException base) used consistently
- Rewrite catalog filtering/search/reporting logic using Java Streams, including one `parallelStream()` benchmark note
- Raw JDBC version that persists products to PostgreSQL — no ORM yet
- NEW: one checkout-adjacent operation (e.g. sending a confirmation) implemented with `CompletableFuture` instead of a blocking call
- Restructure as a Maven build with a `src/test/java` suite of JUnit + Mockito tests
- Replace all console output with SLF4J logging at appropriate levels

Same ShopFlow domain, now backed by real collections, real files, real Streams, real async composition, and a real (if manual) database connection. Phase 3 replaces the manual JDBC with Spring Data.

PHASE 3 OF 5

Framework & API

Spring Boot · REST · Validation · OpenAPI · JWT & OAuth2 · GraphQL · Testing Spring Boot Applications

Wire your OOP and Core Java knowledge into a real framework. Build endpoints one at a time, test each in Postman before moving to the next. Note the reorder: Validation & global exception handling is built first, so REST API Design and everything after it reuses one error-response shape instead of inventing it twice.

Spring Core & Dependency Injection

The container that wires your objects together, so classes declare what they need instead of constructing it themselves.

Subtopics: @Component/@Service/@Repository · @Autowired & constructor injection · the Spring container/ApplicationContext · Controller → Service → Repository layering

1. First Spring Boot App — get the container running end to end

Generate a project with Spring Initializr → add one @RestController → run it and hit it in a browser → add one @Service and inject it

2. Three-Layer Wiring Practice — Controller → Service → Repository from scratch

Write a Repository interface with an in-memory implementation → write a Service depending on it → write a Controller depending on the Service

Study Resources

- Spring Boot Reference Documentation
- Spring Framework: Core Technologies (DI)

Request Validation & Exception Handling [REORDERED]

Rejecting bad input before it reaches your business logic, and turning every failure into one clean, structured response — built once here, and reused by every topic after it, including REST API Design next.

Subtopics: @Valid & Bean Validation annotations · custom constraint annotations · @ControllerAdvice · @ExceptionHandler

1. Validated Registration Endpoint — the standard @Valid workflow

Add @NotBlank, @Email, @Size to a RegisterRequest DTO → annotate the controller param with @Valid → trigger a validation failure and inspect the default error

2. Global Exception Handler — one place for every error, including 404 and 400

Write a @ControllerAdvice class → define a single ErrorResponse DTO (status, message, timestamp) → add @ExceptionHandler for MethodArgumentNotValidException, your custom ShopFlowException, and a catch-all 500 → this becomes the one error shape the whole API returns

3. Field-Level Error Messages — return exactly which field failed and why

Catch MethodArgumentNotValidException in your handler → loop over its field errors → build a Map of field → message → return it in the error response body

Study Resources

- Spring: Validation (Bean Validation)
- Baeldung: Validation in Spring Boot
- Baeldung: @ControllerAdvice and @ExceptionHandler

REST API Design

Designing HTTP endpoints that follow predictable, resource-based conventions other developers can guess without reading docs.

Subtopics: HTTP verbs & status codes · resource naming · DTOs vs entities · pagination & filtering · idempotency for POST/PATCH (NEW) · API versioning strategy (NEW)

1. Todo REST API — the canonical CRUD starter

Build GET/POST/PUT/DELETE for a Todo resource → return correct status codes (200/201/204/404) → add a DTO separate from the entity

2. Product Catalog API — pagination, sorting, filtering

Add GET /products with page and size query params → add sorting and filtering by category → return a wrapped response with total count + page data

3. Idempotency Key Support (NEW) — make a POST safe to retry

Accept an Idempotency-Key header on POST /orders → store seen keys with their response → on a repeated key, return the original response instead of creating a duplicate order → explain why this matters for flaky client networks

Study Resources

- MDN: HTTP Overview
- Spring: Building a RESTful Web Service
- RESTful API Design (restfulapi.net)

API Documentation with OpenAPI/Swagger [NEW]

A REST API without generated, browsable documentation is incomplete by industry standard — this is close to non-negotiable on a real backend team.

Subtopics: springdoc-openapi setup · @Operation/@Schema annotations · Swagger UI · documenting error responses

1. Document the Product Catalog API — generate interactive docs for free

Add springdoc-openapi-starter-webmvc-ui → run the app and open /swagger-ui.html → confirm every existing endpoint from REST API Design already appears

2. Annotate for Clarity — go beyond the auto-generated defaults

Add @Operation(summary=...) to each endpoint → add @Schema descriptions to DTO fields → document the standard ErrorResponse shape from Validation as a reusable response → confirm Swagger UI now reads like real documentation, not a raw method list

Study Resources

- springdoc-openapi Documentation
- OpenAPI Specification
- Swagger UI Documentation

JWT Authentication [UPDATED]

Stateless authentication: the server issues a signed token instead of maintaining a session, and every request proves identity by presenting it — plus the delegated-auth pattern (OAuth2/OIDC) that most companies run in production instead of hand-rolled JWT.

Subtopics: token generation & signing · the Spring Security filter chain · securing endpoints · refresh tokens · CORS configuration (NEW) · OAuth2 / OpenID Connect awareness (NEW)

1. User Auth Service — the full register/login/JWT flow

Build /register hashing the password with BCrypt → build /login verifying credentials → generate a signed JWT on success → build a /me endpoint requiring a valid token

2. Role-Based Endpoint Protection — not just 'logged in' but 'allowed'

Add a role claim to the JWT → protect one endpoint with @PreAuthorize for ADMIN only → confirm a USER token gets a 403

3. CORS Configuration (NEW) — let a real frontend call this API

Add a CorsConfigurationSource bean → allow only your Part B frontend's origin → confirm a browser preflight OPTIONS request now succeeds → confirm an unlisted origin is still rejected

4. OAuth2/OIDC Login Path (NEW) — see the delegated-auth alternative

Register a free OAuth2 client (e.g. via Keycloak run locally, or Spring Authorization Server) → add spring-boot-starter-oauth2-client → wire a 'Login with Google/Keycloak' flow alongside your existing JWT login → write

3 bullet points on when a team would choose OAuth2/OIDC over hand-rolled JWT

Study Resources

- [jwt.io: Introduction to JSON Web Tokens](#)
- [Spring Security Reference](#)
- [Baeldung: JWT with Spring Security](#)
- [Spring Authorization Server Reference](#)

GraphQL

A query language for APIs where the client specifies exactly what shape of data it wants back, in a single request.

Subtopics: schema-first design · queries vs mutations · resolvers/DataFetchers · comparing the feel vs REST

1. GraphQL Notes API — the schema-first starter

Write a `.graphqls` schema for a `Note` type → add a Query for notes and `note(id)` → add a Mutation for `createNote`

2. Product Search as GraphQL — port a REST endpoint over

Design a GraphQL query with filters as arguments, equivalent to your REST product search → implement the resolver reusing your existing `Service`

Study Resources

- [GraphQL.org: Official Learning Guide](#)
- [Spring for GraphQL Reference](#)

Testing Spring Boot Applications

Testing across Spring's layers — from a pure unit test to a full application-context integration test.

Subtopics: @SpringBootTest · MockMvc for controller tests · @DataJpaTest for repository tests · test profiles

1. Controller Test with MockMvc — test an endpoint without a running server

Write a `@WebMvcTest` for one controller → mock the `Service` layer → assert the JSON response on GET → assert 400 on a bad POST body

2. Security-Aware Controller Test — test a JWT-protected endpoint

Write a test hitting a secured endpoint with no token → assert 401 → generate a valid test token and retry → assert 200

Study Resources

- [Spring Boot: Testing Reference](#)
- [Baeldung: Testing in Spring Boot](#)

Final project for this phase — ShopFlow: Backend API Layer

- Product CRUD: POST /products, GET /products/{id}, PATCH, DELETE
- Cart endpoints: add item, remove item, apply coupon, get total
- Order placement: POST /orders, transitions (PENDING → CONFIRMED → SHIPPED), idempotent via Idempotency-Key
- JWT auth: register, login, secure all endpoints except GET /products; CORS configured for the Part B frontend
- Global @ControllerAdvice returning one structured JSON error shape, reused across every endpoint
- GraphQL alternative exposing product search
- NEW: full API documented and browsable at /swagger-ui.html via springdoc-openapi
- NEW: /actuator/health exposed and returning UP (introduced fully in Phase 5)
- Unit + integration tests: service layer with Mockito, MockMvc for controllers, ≥80% coverage on business logic

Extend your Phase 2 core-Java service layer into a full Spring Boot API. Keep the OOP design — now expose it over HTTP, and let Spring Data replace your manual JDBC.

PHASE 4 OF 5

Databases

PostgreSQL · MongoDB · Redis · JPA/Hibernate · N+1 · Query optimization

The phase most people rush and regret later. Hibernate behavior — lazy loading, session scope, N+1 — causes invisible production bugs. Your Phase 2 raw-JDBC work makes it obvious what Hibernate does for you here. This revision adds Redis caching as the natural next layer once indexing and query optimization are second nature.

JPA Entity Mapping & Relationships

Turning Java classes into database tables declaratively, and modeling how those tables relate to each other.

Subtopics: @Entity/@Id/@GeneratedValue · @Column/@Table · @OneToMany/@ManyToOne · @ManyToMany · mappedBy & @JoinColumn

1. One-to-Many: Category → Products — the most common relationship

Add a Category entity → add @OneToMany on Category, @ManyToOne on Product → set mappedBy correctly → save and fetch back

2. Bidirectional Relationship Pitfall — see and fix the classic mistake

Set up a bidirectional relationship without syncing both sides → trigger the inconsistency → fix it with a helper method that sets both sides together

Study Resources

- Spring Data JPA Reference
- Hibernate ORM Documentation
- Baeldung: JPA Entity Mapping

Lazy vs Eager Loading & the N+1 Problem

The Hibernate behavior that causes the most invisible production bugs: how and when related data actually gets fetched.

Subtopics: FetchType.LAZY vs EAGER · detecting N+1 via SQL logs · JOIN FETCH · @EntityGraph

1. Reproduce N+1 On Purpose — see the bug before you fix it

Enable Hibernate SQL logging → fetch 10 orders, each with lazy-loaded items → loop accessing .getItems() → count the queries fired — confirm it's 1+N, not 1

2. Fix With JOIN FETCH and @EntityGraph — compare both approaches

Write a custom @Query using JOIN FETCH → confirm a single query → repeat with @EntityGraph instead → note the difference in approach

Study Resources

- Baeldung: N+1 Problem in Hibernate
- Spring Data JPA: @EntityGraph

JPQL & Native Queries

Writing queries against your entity model (JPQL) or dropping down to raw SQL (native) when you need to.

Subtopics: @Query with JPQL · native SQL queries · named parameters · when to choose each

1. Custom JPQL Finder Methods — beyond what method-name derivation can do

Write a @Query using JPQL for a multi-condition search → use named parameters → compare readability vs a derived method name

2. Pagination in a Custom Query — combine @Query with Pageable

Write a JPQL query taking a Pageable parameter → return a Page → confirm total count and page metadata are correct

Study Resources

- Spring Data JPA: Query Methods
- Baeldung: JPQL vs Native Query

PostgreSQL Indexing & Query Optimization

Making queries fast at scale — understanding what an index actually does and how to prove a query is using one.

Subtopics: B-tree vs GIN indexes · EXPLAIN ANALYZE · composite indexes · when an index doesn't help

1. Baseline Before Optimizing, Then Add an Index

Seed a table with 100,000+ rows → run EXPLAIN ANALYZE and record the baseline → CREATE INDEX on the filtered column → rerun and compare

2. Composite Index for a Multi-Column Query — order matters

Create a composite index on (category, price) → run a query filtering on both → confirm it's used → test filtering on price alone

Study Resources

- PostgreSQL Official Documentation: Indexes
- Use The Index, Luke!

Transactions

Grouping multiple database operations so they either all succeed or all fail together — no partial updates.

Subtopics: @Transactional · propagation levels · rollback rules · isolation basics

1. Transfer Funds End-to-End — the classic transactional example, in Spring

Write a transferFunds(fromId, toId, amount) service method → debit one account, credit another, inside one @Transactional method → force a failure mid-transfer → confirm neither balance changed

2. Propagation: REQUIRES_NEW — a nested transaction that commits independently

Call a REQUIRES_NEW method from within another @Transactional method → fail the outer transaction after the inner one runs → confirm the inner write survived anyway

Study Resources

- Spring: Transaction Management
- Baeldung: Spring @Transactional

MongoDB & Document Databases

A schema-flexible alternative to relational tables — storing nested, denormalized documents instead of normalized rows.

Subtopics: document model (embedding vs referencing) · MongoRepository · custom @Query · aggregation pipelines

1. Embedding vs Referencing — the core MongoDB design decision

Embed a short list of reviews directly inside a Product document → separately model a User referenced only by ID → explain when you'd choose each

2. Aggregation Pipeline: Average Rating — MongoDB's answer to GROUP BY

Write an aggregation pipeline grouping reviews by productId → compute an average rating per product via MongoTemplate

Study Resources

- MongoDB Official Documentation
- Spring Data MongoDB Reference

Caching with Redis [NEW]

Most hot-path reads in a real backend hit a cache long before they touch Postgres. This topic ties directly back into the indexing phase's 'measure before optimizing' instinct — just at the caching layer.

Subtopics: Spring Cache abstraction (@Cacheable/@CacheEvict) · Redis as a cache provider · cache-aside pattern · TTL and eviction strategy

1. Cache the Product Catalog Lookup — the standard cache-aside pattern

Add spring-boot-starter-cache and spring-boot-starter-data-redis → annotate the hot product-lookup method @Cacheable → run docker run redis locally → confirm the second call skips the database (check logs/timing)

2. Cache Eviction on Update — avoid serving stale data

Annotate the product-update method @CacheEvict for the same key → update a product → confirm the next GET reflects the new value instead of a stale cached one → set a TTL as a backstop

3. Before/After Latency Comparison — prove the cache is actually helping

Time 100 repeated requests to the product-lookup endpoint with caching disabled → re-enable caching and repeat → record the latency difference, same format as the earlier EXPLAIN ANALYZE before/after comparison

Study Resources

- Spring: Caching Abstraction

- Redis Documentation
- Baeldung: Spring Boot Redis Cache

Schema Migrations (Flyway)

Version-controlling your database schema the same way you version-control your code, so every environment evolves in lockstep.

Subtopics: versioned migration files · the migrate command · safe vs unsafe migrations · baseline & repair

1. First Flyway Migration — the basic workflow

Add Flyway → write V1__create_products_table.sql → run the app and confirm Flyway applied it automatically

2. Rename Column the Safe Way — a two-step migration to avoid downtime

Add the new column → copy data across in a follow-up migration → drop the old column in a final migration → explain why this is safer than a direct rename

Study Resources

- Redgate Flyway Documentation
- Baeldung: Database Migrations with Flyway

Final project for this phase — ShopFlow: Database Layer

- Normalized PostgreSQL schema: users, products, categories, orders, order_items
- JPA entities with proper relationships and fetch strategies
- Fix N+1 on order history endpoint using @EntityGraph
- Flyway migrations for every schema change from here forward
- MongoDB collection: product_reviews with embedded rating + text, referenced userId
- Index: products.category_id, orders.user_id, orders.created_at; EXPLAIN ANALYZE the 3 most-used queries
- Wrap multi-step operations (checkout, stock transfer) in @Transactional with correct rollback rules
- NEW: @Cacheable product lookups backed by Redis, with @CacheEvict on update and a measured before/after latency comparison
- NEW: repository/integration tests rewritten with Testcontainers, running against a real containerized Postgres instead of H2
- NEW: a docker-compose.yml bringing up Postgres, MongoDB, and Redis together for local development

Connect Phase 3's API to a real PostgreSQL database, add MongoDB for reviews, and Redis for caching — all three now reproducible with one docker-compose up.

PHASE 5 OF 5

Production Readiness & Cloud-Native Basics

Actuator & Observability · Resilience4j · Docker · Messaging Awareness

Entirely new phase. Everything here is scoped to close the gap between 'runs in IntelliJ' and 'runs like a real service' — without turning this roadmap into a full microservices course. Deep distributed-systems topics (service discovery, API gateways, distributed tracing, Kafka at production scale) are intentionally left for a later, dedicated track.

Observability & Actuator [NEW]

Every real Spring Boot service exposes its own health and metrics — this is a small add with an outsized payoff for how 'production-ready' the app feels.

Subtopics: Spring Boot Actuator · /health, /metrics, /info endpoints · Micrometer basics · structured/JSON logging & MDC for request tracing

1. Expose Actuator Health & Metrics

Add spring-boot-starter-actuator → expose /actuator/health and /actuator/metrics → add a custom health indicator checking the database connection

2. Request Tracing with MDC

Add a filter that generates a requestId per incoming request → put it in the SLF4J MDC → confirm every log line for that request now carries the same requestId → switch the log pattern to JSON output

Study Resources

- Spring Boot Actuator Reference
- Micrometer Documentation
- Logback: Mapped Diagnostic Context (MDC)

Resilience & Rate Limiting [NEW]

Once your API calls anything else — a database under load, a third-party service — it needs to fail predictably instead of cascading. Kept deliberately lightweight: awareness of the patterns, not a full resilience framework deep dive.

Subtopics: circuit breaker pattern · retry with backoff · rate limiting · Resilience4j basics

1. Rate-Limit the Product Search Endpoint

Add Resilience4j's RateLimiter to the product-search endpoint → configure a request-per-second cap → confirm requests over the limit get a 429 with a Retry-After header

2. Circuit Breaker Around an External Call

Simulate an unreliable external call (e.g. a mock shipping-quote service that fails ~40% of the time) → wrap it with a Resilience4j CircuitBreaker → confirm the breaker opens after repeated failures and fails fast instead of hanging

Study Resources

- Resilience4j Documentation
- Baeldung: Resilience4j Guide

Containerization with Docker [NEW]

How the app actually gets run outside your IDE. This should happen before Part B begins, so the backend is deployable on its own before a frontend is layered on top.

Subtopics: writing a Dockerfile for a Spring Boot app · multi-stage builds · docker-compose for local dev · environment-based configuration & not committing secrets

1. Dockerize ShopFlow's API

Write a multi-stage Dockerfile (build stage with Maven, slim runtime stage with just the JAR) → build and run the image locally → confirm the API responds on the mapped port

2. Full Local Stack with docker-compose

Extend the compose file from Phase 4 to include the Spring Boot app itself → wire environment variables for DB/Redis/Mongo hosts instead of localhost → bring the entire ShopFlow stack up with one docker-compose up

Study Resources

- Docker Documentation: Dockerfile reference

- Spring Boot: Container Images
- Docker Compose Documentation

Messaging & Events (Awareness Level) [NEW]

Just enough event-driven thinking to recognize the pattern in an interview or a real codebase — deliberately not a full microservices/Kafka-at-scale treatment, which is out of scope for this roadmap.

Subtopics: publish/subscribe basics · the outbox pattern · at-least-once delivery & idempotent consumers · when to reach for Kafka/RabbitMQ vs a simple event

1. In-Process Domain Events — the pattern without the infrastructure

Publish an OrderPlaced application event using Spring's own ApplicationEventPublisher → write a listener that sends a confirmation (reuse your CompletableFuture email-send from Phase 2) → confirm the order-placement logic doesn't need to know the listener exists

2. Outbox Pattern — transactional consistency across a boundary

Add an outbox_events table → in the same @Transactional method that saves an Order, insert an OrderPlaced row into outbox_events → write a simple scheduled poller that reads unprocessed rows and 'publishes' them (a log line is enough) → explain why this avoids the dual-write problem a broker call inside the transaction would have

Study Resources

- Microservices.io: Transactional Outbox Pattern
- Spring: Application Events
- Confluent: Kafka basics (for future reference, not required here)

Final project for this phase — ShopFlow: Production Hardening

- Actuator health/metrics exposed, with a custom DB health indicator
- Every request traceable through logs via a request-scoped MDC id, output as structured JSON
- Rate limiting on the product-search endpoint; a circuit breaker around one simulated external call
- Dockerfile for the API, folded into the Phase 4 docker-compose stack alongside Postgres/MongoDB/Redis
- OrderPlaced modeled as an in-process event, plus an outbox table proving the transactional-consistency pattern

This is the checkpoint where ShopFlow stops being a project that only runs in your IDE and becomes something you could hand to another engineer with a single docker-compose up and a README.

Part A — Backend Engineering — Progress Tracker (Revised)

Check off each topic once you've read its concept sections, built its hands-on projects, and folded the result into ShopFlow where indicated.

PHASE 1 — OOP Fundamentals

- Encapsulation
- Inheritance
- Abstraction
- Polymorphism
- Composition
- Exceptions
- SOLID Principles & Design Patterns
- Records & Sealed Classes (Modern Java) [\[NEW\]](#)

PHASE 2 — Core Java Toolkit

- 2D & 3D Arrays
- Java Collections Framework
- Generics
- Exception Handling — Deep Dive
- File I/O
- Java Streams
- Concurrency & Multithreading [\[UPDATED\]](#)
- JDBC
- Build Tools
- Testing (JUnit & Mockito)
- Logging

PHASE 3 — Framework & API

- Spring Core & Dependency Injection
- Request Validation & Exception Handling [\[REORDERED\]](#)
- REST API Design
- API Documentation with OpenAPI/Swagger [\[NEW\]](#)
- JWT Authentication [\[UPDATED\]](#)
- GraphQL
- Testing Spring Boot Applications

PHASE 4 — Databases

- JPA Entity Mapping & Relationships
- Lazy vs Eager Loading & the N+1 Problem
- JPQL & Native Queries
- PostgreSQL Indexing & Query Optimization
- Transactions
- MongoDB & Document Databases
- Caching with Redis [\[NEW\]](#)
- Schema Migrations (Flyway)

PHASE 5 — Production Readiness & Cloud-Native Basics

- Observability & Actuator [\[NEW\]](#)
- Resilience & Rate Limiting [\[NEW\]](#)
- Containerization with Docker [\[NEW\]](#)
- Messaging & Events (Awareness Level) [\[NEW\]](#)